Adaptive Segment Routing (Challenge EURO/ROADEF 2026): a mixed-integer programming approach

SOICT – Mathematical Modelling project

1 Introduction

Orange operates large IP/MPLS backbone networks, where *traffic engineering* aims at avoiding congestion and keeping a good quality of service, also when some links are unavailable because of failures or planned maintenance. With *Segment Routing* (SR) a packet carries a list of waypoints: between two consecutive waypoints it follows the usual shortest paths, and when several shortest paths exist the traffic is split evenly at every node (ECMP). Choosing waypoints therefore allows to leave the shortest paths and to balance the load without changing the routers' configuration.

The *T-Adaptive Segment Routing* problem (T-ASR) of the Orange challenge is the following. We are given a directed network $G = (V, A)$ with an arc weight $\omega(a)$ and a capacity $c(a)$, a set D of demands $d = (s_d, \tau_d)$ with a traffic volume $\delta(d, t)$ for every time period $t \in T = \{0, \dots, h-1\}$, a maintenance scenario $q(t) \subseteq A$ (arcs that are down at period t), a budget $\beta(t)$ limiting the number of changes of the routing between $t-1$ and t, and a maximum number of segments maxSeg per demand. We must choose an SR path for every demand and every period so that the loads of the links are as balanced as possible. In this work we study periods $T = \{0, 1\}$: period 0 is the nominal state (no budget, no failure), and period 1 has one link down and a budget.

2 Mathematical model

2.1 Data

Network. V is the set of nodes ($n = |V|$), A the set of arcs ($m = |A|$), $\omega(a) > 0$ the weight (field `metric`) and $c(a) > 0$ the capacity (field `capacity`) of arc a; they are read from the `net` file. $G_t = (V, A \setminus q(t))$ is the graph at period t, with $q(t)$ read from the `scenario` file (the links are given by their identifier). **Demands.** D and $\delta(d, t)$ are read from the `tm` file, maxSeg and the budget $\beta(1)$ from the `scenario` file; at $t = 0$ there is no budget.

Split coefficients. Let $FG_t(i, j)$ be the set of arcs lying on a shortest i–j path of G_t. For $i \neq j$ and an arc a, $r(i, j, a, t) \in [0, 1]$ is the fraction of a unit of flow from i to j that crosses a when, at every node, the incoming flow is split evenly among the outgoing arcs belonging to $FG_t(i, j)$ (ECMP). It is zero when $a \notin FG_t(i, j)$ and, by construction, when $a \in q(t)$. We compute r for all pairs (i, j): run Dijkstra's algorithm from every node of G_t to get the distance matrix dist; an arc $a = (x, y)$ is in $FG_t(i, j)$ iff $\text{dist}(i, x) + \omega(a) + \text{dist}(y, j) = \text{dist}(i, j)$; then propagate a unit flow from i over the nodes of $FG_t(i, j)$ sorted by $\text{dist}(i, \cdot)$, splitting evenly. Distances are compared with a relative tolerance of 10^{-9} since some instances have real-valued weights.

Remark. The note `calculating_r.pdf` defines the coefficient instead as the proportion of shortest paths using a, $\sigma_{iu}\sigma_{vj}/\sigma_{ij}$. This differs from the ECMP definition of the problem statement: on the toy instance of the statement, the arc $a_{1,4}$ carries $r(v_0, v_5, a_{1,4}, 0) = 0.5$ under ECMP (Table 2 of the statement) but 2/3 with path counting. The statement's loads (0.3125 on $a_{4,6}$, maximum 0.4375) are reproduced exactly with ECMP only; we therefore use ECMP in all experiments (both variants are implemented).

2.2 Variables

$x_{ij}^{dt} \in \{0,1\}$ for $d \in D$, $t \in T$, $i \in V$, $j \in V \setminus \{i\}$: equal to 1 iff the segment path of demand d at period t contains the segment (i,j). The variable $z \geq 0$ is the maximum link utilization (MLU). For the budget constraint we add $y_{ij}^{dt} \geq 0$ ($t \geq 1$) linearizing $|x_{ij}^{dt} - x_{ij}^{d,t-1}|$.

2.3 Constraints

$$\sum_{j \neq i} x_{ij}^{dt} - \sum_{j \neq i} x_{ji}^{dt} = \begin{cases} 1 & i = s_d \\ -1 & i = \tau_d \\ 0 & \text{otherwise} \end{cases} \qquad d \in D,\ t \in T,\ i \in V \tag{1}$$

$$\sum_{i,j \in V, i \neq j} x_{ij}^{dt} \leq \text{maxSeg} \qquad d \in D,\ t \in T \tag{2}$$

$$\sum_{d \in D} \sum_{i \neq j} r(i,j,a,t)\, \delta(d,t)\, x_{ij}^{dt} \leq c(a)\, z \qquad t \in T,\ a \in A \setminus q(t) \tag{3}$$

$$y_{ij}^{dt} \geq x_{ij}^{dt} - x_{ij}^{d,t-1}, \quad y_{ij}^{dt} \geq x_{ij}^{d,t-1} - x_{ij}^{dt}, \quad \sum_{d, i \neq j} y_{ij}^{dt} \leq \beta(t) \qquad t \in T \setminus \{0\} \tag{4}$$

Constraints (**??**) say that the segments of demand d form a path from s_d to τ_d (a segment (i,j) is a waypoint-to-waypoint leg); (**??**) limits the number of segments (waypoints +1); (**??**) bounds the load of every arc that is up; (**??**) bounds the number of segments that change between two periods, which is exactly $\text{dist}(P^{t-1}, P^t)$. A segment (i,j) such that j is not reachable from i in G_t is fixed to 0.

2.4 Objective

The statement asks for the lexicographic minimization of the sorted vector of all loads. We minimize its first component, the maximum link utilization, with a tiny penalty on the number of segments,

$$\min\ z + \varepsilon \sum_{d,t,i \neq j} x_{ij}^{dt}, \qquad \varepsilon = 10^{-7}. \tag{5}$$

The penalty only prevents useless disconnected cycles that constraints (**??**) would allow; it does not change the optimal value of z beyond ε. The first-stage optimum z^* can be followed by a second stage that fixes $z \leq z^*$ and minimizes the total load, which gives the first two levels of the lexicographic order; we did not use it in the experiments.

3 Numerical experiments

3.1 Instances and implementation

We use the 20 instances of set A ($20 \leq n \leq 400$ nodes, $80 \leq m \leq 2000$ arcs, $20 \leq |D| \leq 6000$ demands), each with two periods, maxSeg = 6, and one link down at period 1 with a budget $\beta(1)$ between 1 and 90. Some instances declare `multigraph: true` without containing parallel arcs, and four instances (05, 08, 12, 14) have real-valued weights; these facts were checked on the data.

The code is written in Julia 1.13 with JuMP and the open-source MILP solver HiGHS, run on a single thread. Environment: Intel Core i7-1270P, 31.6 GB RAM, Windows 11. The CPU time reported is the solve time of the call to `optimize!` only. The solver is warm-started with the solution without any waypoint (all demands on their shortest paths), which is always feasible. Each solution is checked by an independent recomputation of the loads from the waypoints (also exported in the SR-path format of the statement), and the model was validated on the toy instance of the statement against an exhaustive enumeration of the waypoints.

3.2 Protocol

The reported values are averages over the runs performed (different solver seeds). Ten runs are done when the first run finishes within 120 s; otherwise one run is done (column "Runs"). The time limit is 1800 s for period 0 alone and 3600 s for periods 0 and 1 with the budget constraint. The gap of a run that is not optimal is $1 - \mathrm{LB}/\mathrm{UB}$. The model has $|D|\, n(n-1)$ binary variables per period; this exceeds $4.9 \cdot 10^6$ already for instance 06 and $9.5 \cdot 10^8$ for instance 20, so that the model cannot be built in memory for the large instances; they are marked "SKIPPED_TOO_LARGE" in the tables.

3.3 Results at period 0 only (time limit 1800 s)

Inst	n	m	$\lvert D\rvert$	Best obj.	CPU (s)	Status	Gap	Nodes	Runs	Model	Best known
setA-01	20	80	40	0.92938	0.8	Optimal	–	1.0	10	arc-pc	0.92938
setA-02	30	150	45	0.54858	2.3	Optimal	–	1.0	10	arc-pc	0.54858
setA-03	50	250	20	0.94354	5.2	Optimal	–	1.0	10	arc-pc	0.94354
setA-04	50	250	200	0.50105	1801.1	Not Optimal	0.098	0.0	6	arc-pc	0.45912
setA-05	100	396	100	0.11552	291.5	Optimal	–	1.0	10	arc-pc	0.11552
setA-06	100	500	500	0.09859	10.7	Optimal*	–	1.0	1	wp	0.09859
setA-07	100	500	800	0.90799	2.7	Optimal*	–	0.0	1	wp	0.90799
setA-08	150	654	200	0.3189	1.2	Optimal*	–	0.0	1	wp	0.3189
setA-09	150	750	200	0.50538	2.0	Optimal*	–	1.0	1	wp	0.50538
setA-10	150	966	1000	0.06957	300.1	Not Optimal*	0.018	0.0	1	wp-cg-lns	0.06957
setA-11	200	1000	400	0.61354	32.2	Optimal*	–	13.0	1	wp	0.61354
setA-12	200	898	400	0.64008	6.3	Optimal*	–	1.0	1	wp	0.64008
setA-13	200	1000	2000	0.03889	300.2	Not Optimal*	0.001	0.0	1	wp-cg-lns	0.03889
setA-14	250	1108	600	0.43421	0.5	Not Optimal*	0.024	1.0	1	wp-cg	0.43421
setA-15	250	1250	600	0.82521	0.4	Optimal*	–	0.0	1	wp-cg	0.82521
setA-16	250	1452	4800	0.12295	115.7	Optimal*	–	1.0	1	wp-cg	0.12295
setA-17	300	1270	2000	0.42419	2.4	Optimal*	–	1.0	1	wp-cg	0.42419
setA-18	300	1500	2000	0.63337	1.3	Optimal*	–	0.0	1	wp-cg	0.63337
setA-19	300	1998	6000	0.06745	1800.2	Not Optimal*	0.027	632.0	1	wp-cg	0.06745
setA-20	400	2000	6000	0.99131	4.9	Optimal*	–	0.0	1	wp-cg	0.99131

Table 1: Period 0 only. Model `arc-pc`: arc-based MILP of the statement with the split coefficients of `calculating_r.pdf`; time, gap and nodes are averages over the runs listed in column Runs. Rows from the waypoint model (restricted problem, at most one waypoint per demand) are used when the arc-based model does not fit in memory.

3.4 Results at periods 0 and 1 with budget (time limit 3600 s)

Inst	n	m	$\lvert D\rvert$	Best obj.	CPU (s)	Status	Gap	Nodes	Runs	Model	Best known
setA-01	20	80	40	0.92938	39.3	Optimal	–	1.0	10	arc-pc	0.92938
setA-02	30	150	45	0.90307	3012.3	Optimal 4/5	0.2	1.0	5	arc-pc	0.90307
setA-03	50	250	20	0.94354	603.2	Optimal	–	1.0	5	arc-pc	0.94354
setA-04	50	250	200	0.58857	3608.7	Not Optimal	0.51	0.0	2	arc-pc	0.58281
setA-05	100	396	100	0.20499	44.5	Optimal*	–	1.0	1	wp	0.20499
setA-06	100	500	500	0.14085	2223.5	Not Optimal*	–	0.0	1	wp-cg	0.14085
setA-07	100	500	800	0.90799	3.2	Not Optimal*	–	0.0	1	wp-cg	0.90799
setA-08	150	654	200	0.3189	8.5	Not Optimal*	–	1.0	1	wp-cg	0.3189
setA-09	150	750	200	0.84965	0.3	Not Optimal*	–	0.0	1	wp-cg	0.84965
setA-10	150	966	1000	0.1087	300.6	Not Optimal*	–	0.0	1	wp-cg-lns	0.1087
setA-11	200	1000	400	0.78579	1.3	Not Optimal*	–	0.0	1	wp-cg	0.78579
setA-12	200	898	400	0.87987	1.3	Not Optimal*	–	0.0	1	wp-cg	0.87987
setA-13	200	1000	2000	0.07692	302.0	Not Optimal*	–	0.0	1	wp-cg-lns	0.07692
setA-14	250	1108	600	0.53315	78.0	Not Optimal*	–	37.0	1	wp-cg	0.53315
setA-15	250	1250	600	0.8987	1.3	Not Optimal*	–	0.0	1	wp-cg	0.8987
setA-16	250	1452	4800	0.51639	0.0	Not Optimal*	–	–	1	wp-cg-lns-fast	0.51639
setA-17	300	1270	2000	0.42419	527.3	Not Optimal*	–	1.0	1	wp-cg	0.42419
setA-18	300	1500	2000	1.0	17.2	Not Optimal*	–	0.0	1	wp-cg	1.0
setA-19	–	–	–	–	–	no result	–	–	0	–	–
setA-20	400	2000	6000	0.99131	597.5	Not Optimal*	–	0.0	1	wp-cg	0.99131

Table 2: Periods 0 and 1 with the budget constraint. Same conventions as Table **??**.

3.5 Discussion and deviations from the instructions

4 Conclusion